Лабораторна робота №3

Словники. Кортежі. Множини.

Мета роботи: ознайомитись із такими структурами даних як словники, кортежі і множини; вивчити основні методи цих типів даних. Опанувати прийоми розв'язання задач із використанням словників, кортежів і множин.

Теоретичні відомості

1. Словник– *це невпорядкована колекція довільних даних із доступом за ключем.*

Ця структура даних схожа на асоціативні масиви, де дані подаються у вигляді пар

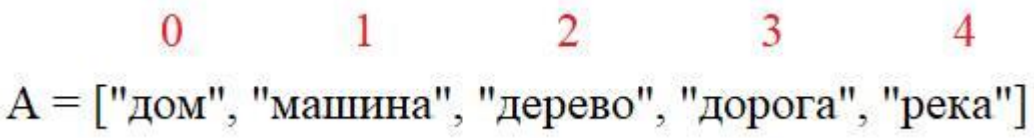
ключ => значення

Наприклад, в ролі ключа можуть виступати звичайні унікальні числа:

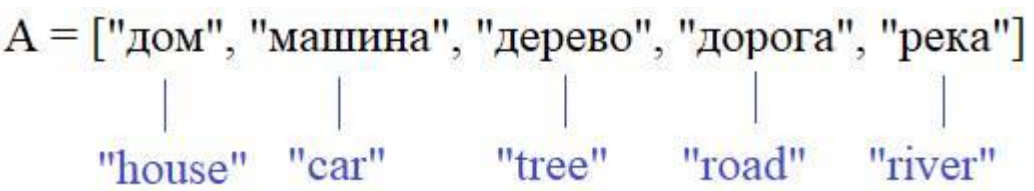
0, 1, 2, …

У цьому випадку ключі нагадують індекси елементів упорядкованого списку:

```
A = ["дом", "машина", "дерево", "дорога", "річка"]
```



Але у списку змінити ці індекси ми не можемо – вони створюються списком автоматично. А що якщо ми хочемо асоціювати ці слова зі словами англійської мови:



Саме тут нам на допомогу і приходить словник. Тобто ми замість ключів 0, 1, 2, … створимо інші – у вигляді англійських слів.

Для створення словника використовується наступний синтаксис:

```
{key1: value1, key2: value2, …, keyN:valueN}
```

І для нашого випадку, маємо:

```
d={"house": "будинок",
   "car":"авто",
   "tree": "дерево",
   "road":"дорога",
   "river":"річка"}
```

Як бачите, ми можемо визначати словник у кілька рядків. Не обов'язково все писати в один рядок.

Тепер ми можемо за ключем отримувати значення цього словника за таким же синтаксисом, що і для списків:

```
d["house"]
```

Отримаємо значення «будинок». Якщо ж вказати ключ, якого не існує, то отримаємо помилку. Зрозуміло, що ключі в словниках завжди унікальні.

Словник ще можна визначити за допомогою функції-конструктора dict:

```
d2 = dict(house = "будинок", car = "автівка",
tree = "дерево", road = "дорога",river = "річка")
```

Тут ключі записуються вже без лапок і після них ставиться знак "=" замість двокрапок. В результаті, отримуємо такий самий словник. І такий спосіб завдання працює тільки, якщо як ключі виступають рядки. Наприклад, з числами такий підхід не працюватиме:

```
d2 = dict(1 = "хата", 2 = "авто")      # SyntaxError
```

Часто в програмах доводиться створювати словники на основі раніше сформованих даних, зокрема списків. Припустимо, у нас є такий список:

```
lst = [[2, "незадовільно"], [3, "задовільно"],
       [4, "добре"], [5, "відмінно"]]
```

Тут у нас перераховані вкладені списки із двома елементами: ключ, значення. Цей список можна перетворити на словник за допомогою функції dict:

```
d3= dict(lst)
```

Об'єкт dict має один цікавий метод:

```
dict.fromkeys(список[, значення за замовчуванням])
```

який формує словник, де ключами будуть елементи списку. Наприклад:

```
a = dict.fromkeys(["+7", "+6", "+5", "+4"])
```

Отримаємо таке:

```
{'+7': None, '+6': None, '+5': None, '+4': None}
```

Усі значення дорівнюють None. Щоб вказати інше значення, використовується другий необов'язковий аргумент:

```
a = dict.fromkeys ([«+7», «+6», «+5», «+4»], «код країни»)
```

отримаємо словник:

```
{'+7': 'код країни', '+6': 'код країни', '+5': 'код країни', '+4': 'код країни'}
```

Якщо потрібно створити порожній словник, то це найпростіше зробити так:

```
d = {}
```

або ж так:

```
d = dict()
```

У всіх наших прикладах у ролі ключів словника виступало чи число, чи рядок. Які типи даних можна ще вказувати? Насправді, будь-які незмінні типи. Зверніть увагу, що саме незмінні! Наприклад, можна написати так:

```
d[True]= "Істина"
d[False]= "Брехня"
```

Тут ключі – булеві значення. В результаті отримаємо словник:

```
{True: 'Істина', False: 'Брехня'}
```

І цей приклад показує, що, присвоюючи словнику значення з новим ключем, воно автоматично додається до словника. В результаті наш спочатку порожній словник став містити два записи. Якщо ж ми наявному ключу присвоюємо інше значення, то він просто змінює своє значення у словнику.

```
d[True] = 1
```

Ось так можна додавати та змінювати значення словника. Тобто, словник відноситься до змінних типів.

Якщо ми спробуємо використати список у ролі ключа, то виникне помилка, бо список – це змінюваний об'єкт.

```
d[[1,2]]= 1
```

На значення словника жодних обмежень не накладається – можуть бути довільні дані:

```
d = {True: 1,
     False: "Брехня",
     "list": [1,2,3],
     5: 5}
```

Для видалення запису із словника за ключем можна скористатися оператором `del`:

```
del d[True]
```

Тепер запису з ключем `True` у словнику не буде. Однак, якщо при видаленні буде вказано ключ, якого у словнику немає, то виникне помилка:

```
deld["abc"]
```

Тому перед видаленням можна перевірити, чи існує даний ключ у словнику:

```
"abc" in d
```

Оператор `in` повертає `True`, якщо ключ є і `False`, якщо відсутній. Або ж, можна робити протилежну перевірку на відсутність ключа:

```
"abc" not in d
```

Як і для будь-якої колекції, ми можемо визначити довжину (число записів) у словнику за допомогою функції `len`:

```
len(d)
```

А також перебрати його значення за допомогою циклу `for`:

```
d = {True: 1, False: "Брехня", "list": [1,2,3], 5: 5}
for x in d:
print(x)
```

І бачимо, що змінна x посилається значення ключів. Але знаючи ключі, ми можемо отримати асоційоване з ним значення:

```
print(x, d[x])
```

2. Кортеж – це впорядкована незмінна колекція довільних даних.

Для завдання кортежу достатньо перерахувати значення через кому:

```
a= 1,2
```

або ж, це еквівалентно такому запису:

```
a=(1,2,3)
```

Щоб присвоїти лише одне значення кортежу, використовується такий синтаксис:

```
b = (1,)
```

тобто, остання кома вказує, що 1 слід сприймати як перший елемент кортежу. Якщо записати просто `(1)`, воно буде сприйматися як звичайне число.

Якщо записати змінні через кому, то їм можна присвоїти значення з кортежу:

```
x, y = (1, 2)
```

Але так:

```
x, y = (1, 2, 3)
```

вже виникне помилка, т.к. число значень у кортежі менше кількості змінних.

Для визначення довжини кортежу (числа його елементів) використовується вже знайома вам функція

```
len(a)
```

Далі доступ до елементів кортежу здійснюється також як і до елементів списків:

- за індексом:

```
a[0]
a[2]
a[-1]
```

- через зрізи:

```
a[1:2]
a[0:-1]
a[:3]
a[1:]
a[:]
```

Про зрізи ми докладно говорили, коли розглядали списки, тож не бачу сенсу дублювати тут цю інформацію. Ось така операція

```
b=a[:]
```

не створює копію кортежу – це будуть посилання на той самий кортеж:

```
print(id(a), id(b))
```

побачимо те саме значення `id`.

Якщо кортежі такі схожі на списки, то в чому їхня перевага перед ними? Інакше кажучи: у яких випадках слід використовувати кортеж замість списку? По-перше, кортеж – це незмінний тип, тобто ми, на відміну від списків, не можемо виконувати такі операції:

```
a[1] = 100
```

це призведе до помилки. Отже, кортеж можна використовувати, коли необхідно "заборонити" програмісту міняти значення елементів списку. По-друге, кортежі можна використовувати як ключі у словників, наприклад, так:

```
d = {}
d[(1, 2)] = "кортеж"
```

По-третє, кортеж зазвичай займає менше пам'яті, ніж такий самий список, наприклад:

```
lst = [1,2,3]
t = (1,2,3)
print(lst.__sizeof__())
print(t.__sizeof__())
```

Як бачите, розмір кортежу менше, ніж у списку. Тут використано метод `__sizeof__`, який повертає розмір даних у байтах.

І, нарешті, по-четверте, операції з кортежами виконуються швидше, ніж зі списками, наприклад перебір елементів або доступ до окремого елемента.

Звідси виходить такий один загальний, глобальний висновок: якщо ми працюємо з незмінним упорядкованим списком, краще використовувати кортеж.

Щоб створити порожній кортеж, можна використовувати такі конструкції:

```
a = ()
b = tuple()
print(type(a), type(b))
```

Постає питання: навіщо створювати порожній кортеж, якщо він відноситься до незмінних типів. Насправді ми лише не можемо змінювати вже існуючі елементи в кортежі, але можемо додавати нові, використовуючи оператор +:

```
a = ()
a = a + (1,)
```

або для додавання на початок кортежу:

```
a = (2,3) + a
```

Також можна додати вкладений кортеж:

```
a += (("a", "hello"),)
```

або робити дублі в кортежі:

```
b = (0,)*10
b = ("hello", "world")*5
```

Усі ці операції цілком допустимі.

Далі, функція `tuple` дозволяє створювати кортежі з будь-яких упорядкованих списків:

```
a= tuple([1,2,3])
a= tuple("Привіт світ")
```

І з кортежу можна сформувати список:

```
t = (1,2,3)
lst = list(t)
```

Кортежі можуть зберігати різні дані:

```
a=(True, [1,2,3], "hello", 5,{"house":"хата"})
```

Причому якщо звернутися, наприклад, до списку, то сам по собі список – це тип, що змінюється, отже, його значення навіть у кортежі ми можемо спокійно змінювати:

```
a[1]
a[1].append("5")
```

Тобто, незмінність кортежу відноситься до його структури елементів та змінних, які посилаються на об'єкти. Але якщо самі елементи кортежу є змінними типами (списки, словники, множини), то це ніхто не забороняє робити. Оскільки сам кортеж – незмінний об'єкт, видалити окремі його елементи неможливо:

```
del a[1]    # TypeError
```

але можна видалити його повністю:

```
del a
```

після цього кортеж a перестане існувати.

Методи кортежів

Кортежі мають метод count, який повертає кількість знайдених елементів із вказаним значенням:

```
a=("abc", 2, [1, 2], True, 2, 5)
a.count("abc")   # 1
a.count(2)       # 2
```

який повертає кількість знайдених елементів із зазначеним значенням. Якщо елемент не знайдено, то повертається число 0:

```
a.count("a")     # 0
```

Наступний метод, повертає індекс першого знайденого елемента із зазначеним значенням.

```
a.index(2)
```

Другий необов'язковий параметр

```
a.index(2, 3)
```

дозволяє задавати індекс початку пошуку елемента. В даному випадку пошук розпочинається з третього індексу. А необов’язковий третій аргумент визначає індекс, до якого йде пошук (не включаючи його):

```
a.index(2, 3, 5)
```

Якщо елемента, який ми шукаємо, не існує, то матимемо помилку:

```
a.index("hello")       # ValueError
```

Завдання:

1. Написати програми у середовищі PyCharm відповідно до номера свого варіанта. (кожне завдання в варіанті по 1.5 бали, тобто максимум за роботу 6 балів).

Варіанти:

1.1 Користувач вводить натуральне число. Знайти усі натуральні дільники цього числа та вивести на екран. Програму реалізувати за допомогою циклу while.
1.2 Вводиться фрагмент URL-адреси у вигляді рядка, наприклад:
`python----loops-for-and--while`.
Необхідно всі дефіси, що йдуть поспіль, замінити одним дефісом (-). Результат вивести на екран.
1.3 Вводиться рядок із текстом англійською мовою. Необхідно його закодувати азбукою Морзе. Після кожної закодованої літери має бути пробіл (символ закінчення коду літери). Після останнього коду пропуску не повинно бути (наприкінці рядка). Після того, як рядок був закодований, треба його розкодувати, *виходячи із закодованого рядка*, тобто отримати оригінальний текст. Словник із кодами абетки Морзе наведено після завдань. На екран вивести закодований і розкодований рядки.
1.4 Вводяться назви міст в один рядок через пробіл. На їх основі формується кортеж. Якщо в цьому кортежі немає міста "Харків", то його слід додати в кінець кортежу. Результат вивести на екран.

2.1 Вводиться натуральне число. Необхідно знайти суму цифр цього числа. Результат вивести на екран. Програму реалізувати за допомогою циклу while.
2.2. Вводиться список міст в один рядок через пропуск. Перетворити цей рядок на кортеж слів і потім через ітератор вибрати перші два значення цього списку і вивести їх на екран.
2.3. З клавіатури вводяться дані у форматі:
`Ім'я_1: номер телефону_1`
…
`Ім'я_N: номер телефону_N,`
доки користувач не введе порожній рядок.
Необхідно на основі введених даних формувати словник, у якому ключами будуть імена, а значення – список номерів телефонів. Імена у списку можуть

повторюватися. Тоді одному імені у словнику буде відповідати кілька номерів. Результат вивести на екран.
2.4. Вводиться рядок, що містить латинські символи, пробіли та цифри. Необхідно виділити з неї всі цифри, що не повторюються (символи від 0 до 9). Вивести на екран знайдені цифри. Якщо таких цифр немає, вивести фразу «унікальні цифри відсутні».

3.1. Вводиться список назв міст в один рядок через пропуск. Визначити, що у цьому списку всі міста мають довжину понад 5 символів. Реалізувати програму з використанням операторів `while` та `break`. Вивести ТАК, якщо умова виконується, і НІ – в протилежному випадку.
3.2. Задається двовимірний список чисел розмірністю 5 х 5 елементів, що складається з нулів і в деяких позиціях одиниць (формується самостійно в коді). Потрібно перевірити, чи не торкаються одиниці одна одну по горизонталі, вертикалі та діагоналі. Тобто навколо кожної одиниці мають бути нулі. Якщо перевірка проходить, вивести фоазу «всі одиниці ізольовані», а якщо ні – то «є одиниці, які контактують».
3.3. Задається словник для перекладу з англійської на українську мову у вигляді (створіть його у коді самостійно):
`d = {'house': 'будинок', 'river': 'річка', 'car': 'машина', ...}`
Користувач вводить рядок з англійськими словами, записаними через пропуск. Необхідно замінити у рядку всі англійські слова на українські, присутні у словнику d. Результат вивести на екран.
3.4. У нічному клубі фіксується перелік гостей. Причому гості можуть виходити з приміщення, а потім знову заходити. Тоді їхні імена фіксуються повторно. На вхід програми надходить такий перелік (кожне ім'я записано з нового рядка). Наприклад:
```
Сергій
Марія
Наталя
…
```
поки не буде введено порожній рядок. Потрібно підрахувати загальну кількість гостей, які відвідали нічний клуб. Вважається, що гості мають унікальні імена. На екран вивести загальну кількість гостей клубу.

4.1. Вводиться список імен студентів в один рядок через пропуск. Визначити, що хоча б одне ім'я в цьому списку починається і закінчується на ту саму букву (без урахування регістру). Реалізувати програму з використанням операторів `while` та `break`. Вивести ТАК, якщо умова виконується, і НІ – у протилежному випадку.
4.2. У деякій країні використовуються грошові купюри номіналом в 1,
2, 5, 10, 50 та 100. Вводиться натуральне число n. Як найменшою кількістю таких грошових купюр можна сплатити суму n? Вивести на екран перелік купюр для формування суми n. Вважайте, що є досить багато купюр усіх типів.

4.3. Є словник з описом предметів:
```
weights = {'олівець': 20, 'дзеркальце': 100,
'парасолька': 500, 'сорочка': 300}
```
Потім у програмі вводяться дані у форматі:
```
предмет_1=вага_1
…
предмет_2=вага_2
```
поки не буде введено порожній рядок. Необхідно введені дані зберегти в окремому словнику d, а потім до словника weights додати вміст словника d. Результат вивести на екран.

4.4. В обліковому записі соціальної мережі Сергія прокоментували фотографію. Деякі відвідувачі залишали кілька коментарів. Потрібно за списком коментарів сформувати словник, ключами якого будуть імена коментаторів, а значеннями – кортежі коментарів (тобто кортежі рядків). Коментарі надходять на вхід програми у форматі:
```
ім'я_1: 'коментар_1'
ім'я_2: 'коментар_2'
...
ім'я_N: 'коментар_N'
```
(доки не буде введено порожній рядок).

Також вважається, що імена у різних коментаторів не збігаються. Вивести на екран отриманий словник і загальну кількість унікальних коментаторів.

5.1. Користувач вводить натуральне число n. Перевірити, чи є це число простим (число називається простим, якщо воно не ділиться ні на які натуральні дільники окрім себе і одиниці). Реалізувати програму з використанням операторів `while` та `break`.

5.2. Задається список довільних натуральних значень в рядок через пробіл і натуральне число n на окремому рядку. Необхідно створити новий двовимірний список, рядок якого має довжину n і який би містив усі значення заданого списку. Результат вивести на екран. Якщо n не є дільником довжини вихідного списку, то останній рядок треба доповнити нулями так, щоб утворився прямокутний двовимірний список. Наприклад, якщо вихідні дані бути такими:
```
2 3 6 7 4 2 5 7 8 2
4
```
то результат має виглядати так:
```
[[2, 3, 6, 7], [4, 2, 5, 7], [8, 2, 0, 0]]
```
5.3. Вводиться рядок з українськими та латинськими літерами. Наприклад: `"Заняття з мови Python"`. Необхідно за допомогою словника (транслітерації) перевести введений рядок у латиницю. Крім того, символи `" ?!;:,"` замінювати на символ дефісу (-). Заміни робити без урахування регістру (рядок перевести в нижній регістр – малі літери). Результат вивести на екран.

5.4. Вводяться прізвища студентів в один рядок через пропуск. На основі цього рядка формується кортеж. Відобразіть на екрані всі прізвища цього кортежу, які містять фрагмент "ко" (без урахування регістру). Імена виводяться в один рядок через пробіл.

6.1. Вводяться цілі числа в один рядок через пробіл. Необхідно перетворити ці дані на список цілих чисел. Потім перебрати цей список в циклі `for` і підсумувати всі непарні значення. Результат вивести на екран.
6.2. Задано ігрове поле для гри «хрестики-нулики» у вигляді двовимірного списку. Наприклад:

```
P = [['x', 'x', 'o'],
     ['o', 'x', 'x'],
     ['#', 'x', '#']]
```

Тут 'x' – хрестик; 'o' - нулик; '#'-вільна клітина. Необхідно перевірити, чи є у полі P виграшна ситуація для хрестиків. Вивести результат аналізу ігрової ситуації: «хрестики перемогли», «хрестики програли» або «боротьба триває».
6.3. Вводиться інформація про книги у форматі:

```
автор_1: назва_1
…
автор_N: назва_N
```

доки не буде введено порожній рядок. Автори можуть повторюватись. Необхідно сформувати словник тільки з унікальних авторів з першим значенням, що зустрілося. Результат вивести на екран.
6.4. Вводяться назви міст в один рядок через пропуск. На основі формується кортеж. Якщо в цьому кортежі є місто, назва якого починається і закінчується на голосну (напр., «Осло», «Абу-Дабі»), то слід його видалити. Результат вивести на екран у вигляді рядка із назвами міст через пробіл. Зверніть увагу, що назва, яку ви видаляєте, може бути записана кілька разів. Тоді треба видалити всі її згадки.

7.1. Вводиться список назв міст в один рядок через пропуск. Перебрати всі ці назви за допомогою циклу `for` і визначити, чи починається назва наступного міста на останню літеру попереднього міста в список. Якщо останньою виявилась буква «ь», то береться наступна з кінця буква слова. Вивести на екран фразу «умова виконана», якщо послідовність задовольняє правилу, або «умова не виконана» в протилежному випадку.
7.2. Є список предметів із зазначенням їхньої ваги (у грамах) у форматі:

```
олівець 20
люстерко 100
парасолька 500
сорочка 300
молоток 600
пила 400
вудка 1200
```

Їх слід подати у вигляді списку кортежів, де кожний кортеж – впорядкована пара виду (предмет, вага). Вводиться натуральне число m – сумарна вага, яку можна покласти у рюкзак. Предмети кладуться в порядку зменшення їх ваги. Напишіть програму, яка для введеного значення m визначить найбільшу кількість предметів, покладених у рюкзак. На екран вивести список цих предметів, їхню кількість і залишкову вагу, яка не була використана. Кожен предмет може з'явитись у єдиному екземплярі.
7.3. Вводяться оцінки студента в один рядок через пробіл. Необхідно сформувати словник, у якому ключами є оцінки (числа), а значення – кількість цих оцінок у введеному списку. Результат вивести на екран.
7.4. Вводяться два списки цілих чисел кожен із нового рядка (у рядку набори чисел через пропуск). Необхідно вибрати і відобразити на екрані цифри, які є у першому списку, але відсутні в другому. Результат виведіть на екран у вигляді рядка чисел, записаних через пробіл. Завдання вирішити з використанням множин.

8.1. Вводиться рядок із номером телефону. Очікується формат введення: «+3(xxx)xxx-xx-xx», де x - це цифра. Необхідно перевірити, що введений рядок відповідає цьому формату. Вивести фразу «номер коректний», якщо відповідає, і «номер введено із помилками» в протилежному випадку.
8.2. Вводиться чотирицифрове натуральне число. Подумайте, як можна визначити ітератор перебору його цифр. Виведіть цифри введеного числа за допомогою ітератора.
8.3. Вводяться числа в один рядок у форматі:
`1; 3 4 5; 2 4 3 4; 1; 6 8 10; …`
Таким чином, вихідні дані є групами чисел, розділених крапкою з комою. Одне число – це довжина відрізка, три – це сторони трикутника, чотири – сторони чотирикутника, тощо. На основі введених даних необхідно сформувати словник із ключами: «відрізок», «трикутник», «чотирикутник» тощо, а значеннями має бути двовимірний (вкладений) кортеж, що містить відповідні групи чисел. Вважайте, що вихідні дані не містять груп по два числа і група не може мати більше, ніж 6 чисел. Вивести на екран вміст словника.
8.4. Вводяться два списки міст кожен із нового рядка (у кожному рядку назви через пробіл). Необхідно порівняти їх між собою на рівність по унікальних (не повторюваних) містах. Якщо списки містять одні й самі унікальні міста, то вивести «ТАК», інакше – «НІ». Завдання вирішити з використанням множин.

9.1. Вводиться список у вигляді цілих чисел в один рядок через пропуск. Спершу необхідно сформувати список на основі введеного рядка, а потім, кожне значення цього списку помножити на його індекс в цьому списку. Відобразити результат на екрані. Програму слід реалізувати за допомогою функції `enumerate`.
9.2. Іван Іванович 1 березня відкрив рахунок у банку, поклавши x грн. Кожного місяця розмір вкладу збільшується на y% наявної суми (x і y вводить

користувач). Визначити через скільки місяців розмір вкладу перевищить n грн. Результат (кількість місяців) вивести на екран.
9.3. Вводяться номери телефонів у форматі:
+3xxxxxxxxxx
+6xxxxxxxxxx
+3xxxxxxxxxx
+5xxxxxxxxxx
…
поки не буде введено порожній рядок. Тут x – це цифри. Необхідно сформувати словник, у якому ключами будуть коди країн: +3, +5, +6, +8, …, а значеннями – список відповідних номерів телефонів (без коду країни).
9.4. Вводяться два списки міст кожен із нового рядка (у рядку назви через кому із довільною кількістю пробілів), які об'їхав Сергій у 1-й 2-й роки своєї подорожі Україною. Потрібно визначити, чи включав його маршрут 2-го року усі міста маршруту 1-го року? Якщо це так, то вивести «ТАК», інакше – «НІ». Завдання вирішити з використанням множин.

10.1. Вводиться список у вигляді цілих чисел в один рядок через пробіл. За допомогою циклу `for` необхідно знайти найменше парне значення цього списку. Отриманий результат вивести на екран. Якщо парного значення немає, вивести слово "None" (без лапок). Реалізувати програму без використання функції `min`.
10.2. Користувач вводить натуральне число. Необхідно перевірити, чи є в цьому числі якісь дві однакові цифри, що стоять поруч. Вивести всі такі цифри, які повторюються у наведений спосіб, якщо такі є, інакше вивести «однакових цифр, що стоять поруч, не знайдено».
10.3. У програмі в кожному новому рядку вводяться цілі числа, доки не буде введено число 0. Необхідно для кожного введеного числа обчислити косинус кута, градусна міра якого виражається введеним числом, і вивести результат з точністю до тисячних. Отриманий результат слід також зберегти у словнику, де ключ – градусна міра кута, а значення – його косинус. Якщо числове значення вводиться повторно, то його обчислювати не потрібно, а брати обчислене раніше значення із словника.
10.4. Вводяться два списки цілих чисел кожен із нового рядка (у рядку набори чисел через пробіл). Необхідно вибрати та відобразити на екрані числа, присутні в першому та другому списках. Результат виведіть на екран у вигляді рядка чисел, записаних через пробіл. Завдання вирішити з використанням множин.

Додаток:

1. Словник для кодування кирилиць і символу пробілу азбукою Морзе:

```
morze = {'A':'.-', 'B':'-...', 'C':'-.-.', 'D':'-..', 'E':'.', 'F':'..-.', 'G':'--.', 'H':'....', 'I':'..', 'J':'.---', 'K':'-.-', 'L':'.-..', 'M':'--', 'N':'-.', 'O':'---', 'P':'.--.', 'Q':'--.-', 'R':'.-.', 'S':'...', 'T':'-', 'U':'..-', 'V':'...-', 'W':'.--', 'X':'-..-', 'Y':'-.--', 'Z':'--..'}
```

Зміст звіту

1. Титульний лист із назвою лабораторної роботи, номером свого варіанту, прізвищем студента та групи.
2. Мета лабораторної роботи.
3. Завдання.
4. Лістинг програми (скопійований з IDE).
5. Результати роботи програми (скріншоти).